Manual Migrations and Downgrades

- Due No Due Date
- Points 1
- Submitting a website url

FORK (https://github.com/learn-co-curriculum/python-p3-manual-migrations-and-downgrades/fork) (https://github.com/learn-co-curriculum/python-p3-manual-migrations-and-downgrades) (https://github.com/learn-co-curriculum/python-p3-manual-migrations-and-downgrades/issues/new)

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
- Manage database tables and schemas without ever writing SQL through Alembic.
- Use SQLAlchemy to create, read, update and delete records in a SQL database.

Key Vocab

- **Schema**: the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist**: save a schema in a database.
- **Engine**: a Python object that translates SQL to Python and vice-versa.
- **Session**: a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction**: a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration**: the process of moving data from one or more databases to one or more target databases.

Introduction

In the last lesson, we started working with Alembic to generate and carry out migrations, or changes to the database schema. Alembic is a powerful tool when used with the SQLAlchemy ORM, and it can generate migrations that account for many of the common changes we might make to a database schema:

- Creating and dropping tables.
- Creating and dropping columns.
- Most indexing tasks.
- Renaming keys.

That being said, there are certain tasks that Alembic can help us with but cannot carry out on its own:

- Table name changes.
- Column name changes.
- Adding, removing, or changing constraints without explicit names.
- Converting Python data types that are not supported by the database.

In this lesson, we will explore writing manual migrations and how to roll back, or **downgrade**, migrations that were unnecessary or went awry.

Building a Migration Manually

Alembic can't detect changes to table names, so let's practice writing manual migrations by changing the `students` table to `scholars`. We can do this very easily in SQLAlchemy through changing the value of the `__tablename__` class attribute in `models.py`:

```
# models
# path, imports, engine, base

class Student(Base):
    __tablename__ = 'scholars'
    ...
```

Next, we will generate a migration from the command line:

```
$ alembic revision -m "Renaming students to scholars"
  Generating .../python-p3-manual-migrations-and-downgrades/P3/migrations/versions/91381a2f4148_renaming_students_to_sch
```

Let's navigate to our new migration file in the `migrations/versions/` directory and add some functionality:

```
# 91381a2f4148_renaming_students_to_scholars.py

def upgrade() -> None:
    op.rename_table('students', 'scholars')


def downgrade() -> None:
    op.rename_table('scholars', 'students')
```

This tells Alembic to change the table name upon upgrade, but also to change it back upon downgrade past this migration. We can run this migration with the same command that we used for autogenerated migrations:

```
$ alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade 361dae855898 -> 91381a2f4148, Renaming students to scholars
```

Double check that your table name has changed, either through VSCode's SQLite Viewer extension or the `sqlite3` command from the command line:

```
$ sqlite3 migrations_test.db
SQLite version 3.37.0 2021-12-09 01:34:53
Enter ".help" for usage hints.
sqlite> .tables
alembic_version  scholars
```

It looks like there's a `scholars` table where `students` used to be- success!

Alembic provides a number of helpful database management operations through the **`op` module** (https://alembic.sqlalchemy.org/en/latest/ops.html). Remember to check the documentation if you're seeking out specific functionality- Alembic might already be able to do it for you.

Checking Migration Level

Before carrying out your own migrations, it's always best to check which migrations have already been applied to the database. You can find the last migration applied by using the `alembic current` command from the command line. This will return the ID of the current migration, as well as information on whether it is the most recent migration, or head. (Remember that migrations are only pushed to the database when `upgrade` is run!)

You can also see the full history of migrations applied to the database with the `alembic history` command.

Downgrading Migrations

While `scholars` is a lovely name for a table, it is not necessarily the best descriptor for a 4th grader. We also used the word "student" in many of the column names- I think we might have a problem.

To downgrade migrations, we need to find the ID of the migration that we want to return to. This would work even if we chose to return to the original empty base migration- but let's search for the original `students` table instead. For me, that ID is `361dae855898`. It is very likely that your migration's ID is different- you can find this ID in the name of the migration file in `migrations/versions/` or by parsing through the output from `alembic history`.

Once you have found the correct revision ID, all you need to do is run the `alembic downgrade` command:

```
$ alembic downgrade 361dae855898
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running downgrade 91381a2f4148 -> 361dae855898, Renaming students to scholars
```

Now we can check that your table name has changed again, either through VSCode's SQLite Viewer extension or the `sqlite3` command from the command line:

```
$ sqlite3 migrations_test.db
SQLite version 3.37.0 2021-12-09 01:34:53
Enter ".help" for usage hints.
sqlite> .tables
alembic_version  students
```

> **Note**: Alembic will *not* force you to write perfect migrations before carrying them out, so it is important to think about what your code will do before running your first manual upgrade.

Instructions

If you have not already, run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment.

- Rename a column in the `Student` model.
- Manually generate a migration using Alembic.
- Upgrade your database schema with `alembic upgrade head`.
- Revert your change with `alembic downgrade [revision_ID]`.

Once your database schema has been upgraded and downgraded, commit and push your work using `git` to submit.

Conclusion

You should now have a basic idea of how to make all variety of changes to database schemas using SQLAlchemy and Alembic. Next, let's discuss how to fill our databases a little more efficiently than we have so far.

Resources

- SQLAlchemy ORM Documentation (https://docs.sqlalchemy.org/en/14/orm/)
- SQLAlchemy ORM Column Elements and Expressions (https://docs.sqlalchemy.org/en/14/core/sqlelement.html)
- Tutorial - Alembic (https://alembic.sqlalchemy.org/en/latest/tutorial.html)
- Operation Reference - Alembic (https://alembic.sqlalchemy.org/en/latest/ops.html)